

Bug Prediction Models: seeking the most efficient

Ingrid Marçal

`ingrid.marcal@unesp.br`

São Paulo State University

Rogério Eduardo Garcia

São Paulo State University

Research Article

Keywords: Bug Prediction, Model choice, Ensemble models, Bugs

Posted Date: January 30th, 2024

DOI: <https://doi.org/10.21203/rs.3.rs-3900175/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Bug Prediction Models: seeking the most efficient

Ingrid Marçal and Rogerio Eduardo Garcia[†]

Department of Computer Science, São Paulo State University, Presidente Prudente, São Paulo, Brazil.

Contributing authors: ingrid.marcal@unesp.br; rogerio.garcia@unesp.br;

[†]These authors contributed equally to this work.

Abstract

Choosing the most appropriate machine learning model for bug prediction tasks is critical. **This paper primarily compares the predictive power of individual models versus ensemble models.** We begin by experimenting with popular single-machine learning models commonly used in bug prediction, like neural networks and support vector machines. Additionally, we test with ensemble models that combine individual models' unique strengths, aiming to maximize each's benefits. Our evaluation is based on datasets containing historical development data from well-known open-source software projects. We rely on various metrics when assessing the models, encompassing accuracy, precision, recall, and F1 score. Based on our research findings, it has been observed that ensemble models tend to outperform single models, particularly when it comes to maintaining resilience across various datasets. Nevertheless, factors like the project's complexity, data availability, and computational resources all play a role in determining whether to use single or ensemble models. This paper offers a thorough analysis of the factors to consider when selecting machine learning models and approaches for bug prediction, providing valuable insights into the field. Furthermore, it offers practical advice for professionals, enabling them to make informed choices.

Keywords: Bug Prediction, Model choice, Ensemble models, Bugs

1 Introduction

Bugs have been a matter of research interest in software engineering for a long time, primarily due to their role as a critical indicator of software quality (Ma et al, 2020; Jahanshahi et al, 2020). The ever-evolving challenges in Software Engineering enhance

this research interest. As systems increase in size and complexity, the ability to rapidly discover and address bugs becomes technically demanding and increasingly critical.

While finding bugs may be technically tricky, it is essential for maintaining high-quality software development in a fast-paced environment (Martinez et al, 2023).

Bug prediction aims to determine the areas of the source code that are more likely to have bugs (Shan et al, 2014; Habibi et al, 2018). The bug prediction results can guide refactoring the identified defective code areas and reduce software development costs (Nagappan and Ball, 2005; Zimmermann et al, 2007; Tantithamthavorn and Hassan, 2018).

Despite its functional utility, bug prediction can be challenging, often requiring complex analysis and insights into software patterns (Arora et al, 2015; Kamei and Shihab, 2016; Bhandari et al, 2022). As a result of this complexity, the usage of AI techniques for bug prediction has grown, signaling a substantial shift in how bug identification is conducted. (Wangoo, 2018; Tantithamthavorn and Jiarapakdee, 2021; Bommi and Negi, 2023; Sofian et al, 2022). Exploration of machine learning models to predict bugs in source code takes advantage of AI’s ability to analyze large and complex software datasets (Shan et al, 2014; Habibi et al, 2018; Soe et al, 2018; Rahim et al, 2021; Aleithan, 2021; Shen et al, 2022).

Several machine learning models have been proposed for bug prediction, including single models (Shan et al, 2014; Habibi et al, 2018; Soe et al, 2018; Rahim et al, 2021; Aleithan, 2021; Shen et al, 2022) and ensemble models (Rathore and Kumar, 2017; Alazba and Aljamaan, 2022; Johnson et al, 2022).

Single models are those built using a particular algorithm to make predictions or analyze data (Chung and Teo, 2023). This type of model is developed utilizing a single algorithm, such as decision trees (Yang, 2019), linear regression (Maulud and Mohsin Abdulazeez, 2020), or a neural network (Ajit et al, 2020). Ensemble models result from a strategy combining the predictions of several machine learning algorithms to achieve more accurate predictions than any single model (Chung and Teo, 2023).

While single models are effective in predicting bugs, ensemble models can potentially improve prediction results even further (Ali et al, 2022). It is necessary to emphasize that any bug prediction approach does not consistently outperform others in every domain. However, it remains important to determine which category of algorithms is most effective for a specific task environment (Wolpert and Macready, 1997).

When working with bug prediction, understanding the prediction model and the task at hand is crucial (Saharudin et al, 2020). While single models are versatile and can be used in various applications such as image recognition (Zhang et al, 2021; Luo and Luo, 2023) and natural language processing (Varshitha et al, 2023), their application to bug prediction is particularly noteworthy. The landscape of bug prediction also sees a significant use of ensemble models.

According to Nisbet et al. (Nisbet et al, 2017), one of the motivations for using ensemble models is to reduce the generalization error of the predictions. When the selected models for the ensemble are diverse and independent, it is possible to minimize prediction errors with the combination of their results. Integrating diverse models

allows ensemble approaches to provide more accurate and robust results for bug prediction because ensemble models leverage the collective strengths of several algorithms to improve the overall prediction capability (Ali et al, 2022).

This paper aims to provide evidence-based guidance on selecting the most appropriate model type for bug prediction, whether a single or ensemble model. We define the following research questions:

- **RQ1:** How does the performance of single and ensemble models compare for specific contexts?
- **RQ2:** Do ensemble models outperform single models for defect prediction regarding accuracy, precision, recall, F1 score, and AUC?
- **RQ3:** What are the trade-offs between single and ensemble models for defect prediction?

To answer the defined research questions, we use Apache Commons Projects as our testbed and thoroughly evaluate single and ensemble models to predict bugs. This evaluation is grounded in the key performance metrics of precision, recall, F1-score, Accuracy, and AUC. Our approach is designed to compare the bug prediction effectiveness of single and ensemble models and to understand the nuances that would inform a practitioner’s decision in choosing these approaches for optimal bug prediction.

Our results indicate that ensemble models consistently exhibit slightly better bug prediction performance than single models, regardless of the dataset used. This consistent edge in performance underscores the potential of ensemble approaches to enhance the efficacy of bug predictions, which could contribute to developing more reliable software systems.

Our findings will equip practitioners with critical insights into the performance characteristics of single and ensemble models in the context of bug prediction. This research is about theoretical comparisons and offering concrete, data-driven guidance to inform real-world decision-making. Practitioners can leverage our results to make informed choices about which model types to deploy, depending on their specific needs and the nature of their software projects.

The remainder of this paper is organized as follows: Section II, ‘Background and Related Work,’ delves into the existing literature and provides a foundational understanding of the current state of AI models in bug prediction, highlighting both single and ensemble approaches. In Section III, ‘Methodology,’ we begin by defining our research questions, which aim to guide the investigation towards our objectives. Also, we describe the research design, data sources, and analytical methods employed in our study. Section IV, ‘Results,’ presents the findings of our analysis, detailing the performance of single and ensemble models across various datasets. Section V, ‘Discussion,’ interprets these findings, discussing their implications and how they align with or challenge existing knowledge. Section VI, ‘Threats to Validity,’ critically examines potential limitations and biases in our study, ensuring a transparent evaluation of our research. In Section VII, ‘Future Work,’ we outline prospective directions for further research, building upon the insights gleaned from our study. Finally, Section VIII, ‘Conclusions,’ summarizes the key contributions of our work, emphasizing its

practical implications for practitioners in software engineering and the broader field of AI-driven bug prediction.

2 Background and Related Work

Bug prediction has emerged as a prominent area of interest within software engineering, owing to its potential to facilitate the identification of software vulnerabilities. Machine learning-based models have become increasingly prevalent for bug prediction due to the model’s ability to provide precise and accurate evaluations of software characteristics, enabling the prediction of potential bugs.

Initial research in software defect prediction was **primarily focused on evaluating various machine learning models to enhance the precision and effectiveness** of bug prediction (Pachouly et al, 2022). Extensive research has been conducted with Bayesian algorithms, comprising a range of statistical models based on Bayes’ theorem (Turhan and Bener, 2009; Dejaeger et al, 2013; Okutan and Yildiz, 2014). These algorithms are Naive Bayes, Bayesian Belief Network (BBN), and Bayesian Network. The Support Vector Machine (SVM) is a popular machine-learning technique that employs similarity measures to predict bugs. The K-means and Hierarchical Clustering algorithms effectively group data based on shared traits. By organizing the data into clusters with high commonality, these algorithms enable the identification of areas of code where software bugs are likely to occur.

Prior research has demonstrated that no single classifier consistently outperforms others, as their effectiveness depends on the specific dataset under consideration (Fernández-Delgado et al, 2014; Caruana and Niculescu-Mizil, 2006).

When applied to Bug Prediction, ensemble techniques combine different classifiers to improve bug prediction performance. Tosun et al. (Tosun et al, 2008) conducted an experimental study on the Validation and Voting technique, which involves aggregating the outputs of various classifiers using a specific aggregating function. In this case, the aggregation function relies on a majority voting scheme, whereby a class is classified as buggy if most models predict it. This approach is effective in practice, improving the performance of prediction models (Jacob et al, 2021). In contrast, the classification task is hindered when most models fail to predict bugginess.

In this work, we aim to investigate the performance of single and ensemble models when applied to bug prediction in open-source systems. We experiment with seven well-known single machine learning algorithms, CART (Breiman et al, 1984), KNN (Cover and Hart, 1967), LDA (Fisher, 1936), LR (Hosmer et al, 2005), NB (Cestnik, 1990), and SVM (Cortes and Vapnik, 1995). In addition, we analyze two ensemble models, RF (Breiman, 2001) and voting classifiers (Dietterich, 2000).

3 Methodology

This section describes the methodology we used to execute and compare the outcomes of single-model prediction with those of the ensemble model in the context of the chosen subject systems. We defined a comprehensive methodology encompassing several stages: subject systems selection, data collection, model selection, training, ensemble

model construction, and evaluation metrics. These stages ensure the research findings’ reliability, validity, and reproducibility.

The choice of subject systems stage involves carefully identifying and selecting relevant systems that align with the research objectives. The data collection stage involves the systematic gathering of metrics from the development history of each subject system, which is then used to train the models. The model selection stage involves identifying and selecting appropriate models best suited to the research objectives. The training stage involves the iterative process of refining the models using training data. The ensemble model construction stage consists of integrating multiple models to improve the accuracy and robustness of the predictions. Finally, the evaluation metrics stage involves using appropriate metrics to assess the validity of the model’s performance and the research findings.

Overall, we defined this methodology to provide a rigorous and systematic approach to conducting experiments with different bug prediction algorithms. The following sections describe each phase of the explained methodology.

3.1 Subject Systems

The availability of an appropriate dataset is crucial for conducting an adequate analysis of defect prediction models. In this study, we chose projects from Apache Commons to generate the necessary datasets. The selected projects are *commons-cli* and *commons-compress*.

These Apache Commons projects are well-established software components widely adopted in the open-source community and offer a substantial code base with extensive historical information (Table 1). Also, they represent different types of functionality and domains, making them suitable for evaluating the performance of defect prediction models across different software contexts.

The selection systems’ diversity allows for exploration by the prediction model of potential variations in defect characteristics, coding styles, and software development practices, which can further enhance its generalizability.

Table 1 Subject Systems

Subject	Number of Commits	Approximate Size (LOC)
commons-cli	1.285	11.495
commons-compress	4.077	232.447

3.2 Data Collection

Obtaining datasets that are tailored to the specific needs of one’s research can be a challenging task. To address this issue, we have developed our datasets through a meticulous data collection process encompassing a range of software metrics. Developing our datasets ensures their content is comprehensive, accurate, and well-suited to our research needs. In this study, we collected metrics from the software repository

of each subject system. The selected metrics aim to ensure relevant information about the systems under investigation and to allow answering the defined research questions. The metrics were collected from the software repository of each subject system in the Apache Commons. They are:

- **Number of authors:** The number of authors contributing to a file provides a measure of collaboration and the diversity of expertise involved in its development. Research has shown that defects can be influenced by the characteristics and experience levels of developers (Qu et al, 2021). Considering this metric, we aim to explore the relationship between team dynamics and defect proneness.
- **Age:** The age of a file, measured as the time elapsed since the last commit, reflects its stability and maturity. Older files may have undergone more testing and refinement, potentially reducing their defect-proneness. Conversely, recent modifications may make newer files more prone to defects. Including age as a metric allows us to investigate the impact of file maturity on defect prediction.
- **Number of unique changes:** The count of unique changes made to a file captures the level of modification and activity surrounding it. Files that have undergone frequent changes may exhibit higher defect densities, as each change introduces the potential for new defects. By considering this metric, we aim to capture the volatility of files and their relationship with defect occurrence.
- **Size:** File size, measured in lines of code, is a widely studied metric in defect prediction. Larger files tend to be more complex and may contain more opportunities for defects to arise. We aim to investigate the relationship between code complexity and defect proneness by including size as a metric.
- **Lines added and Lines deleted:** The number of lines added and deleted from a file reflects the modification and maintenance activities surrounding it. Files with many lines added or deleted may undergo significant changes that can introduce defects. Including these metrics allows us to capture the impact of code modifications on defect prediction.

By selecting these metrics, we aim to comprehensively understand the factors contributing to defect occurrence in the Apache Commons systems. Their inclusion allows us to explore various dimensions of software development, such as team dynamics, code evolution, complexity, and maintenance activities, and their influence on defect proneness.

The chosen metrics were obtained by executing git commands on each repository and subsequently processing the results to extract the corresponding numeric values. For example, the "number of authors" metric refers to the sum of distinct commit authors of a given file.

The target variable in our dataset corresponds to whether a specific file in a particular commit is classified as buggy or not. To determine the target variable, we utilized the SZZ algorithm (Borg et al, 2019), which helped identify the files marked as buggy in each commit. During the training phase, the target variable was not used as a feature for model training. However, it was employed to validate the results obtained from applying each model, allowing us to assess the accuracy and effectiveness of the models in predicting the buggy files in different commits.

3.3 Model Selection

Selecting suitable machine learning models is an important activity in defect prediction research, as it directly influences the precision and efficacy of the prediction results. In this work, we selected various models, which were subsequently assessed for their effectiveness in predicting defects within the Apache Commons systems. The models chosen for the study are:

- **CART (Classification and Regression Trees)**: CART is a decision tree-based model that recursively partitions the data based on feature values to form a tree-like model. It is known for its simplicity, interpretability, and ability to handle categorical and numerical data. By including CART, we can assess decision tree-based models' predictive power and interpretability in defect prediction ([Breiman et al, 1984](#)).
- **KNN (k-Nearest Neighbors)**: KNN is a non-parametric model that classifies instances based on their proximity to labeled instances in the training data. It is particularly suitable for identifying patterns in data without assuming any underlying distribution. By incorporating KNN, we aim to evaluate the performance of instance-based learning approaches in defect prediction ([Cover and Hart, 1967](#)).
- **LDA (Linear Discriminant Analysis)**: LDA is a statistical model that aims to find linear combinations of features that can discriminate between classes. It assumes that the data follows a Gaussian distribution and is commonly used for dimensionality reduction and classification tasks. By including LDA, we can explore the efficacy of linear discriminant-based models in defect prediction ([Fisher, 1936](#)).
- **LR (Logistic Regression)**: LR is a widely used statistical model that estimates the probability of an instance belonging to a specific class. Using a logistic function, it models the relationship between the features and the binary response variable. By incorporating LR, we can evaluate the effectiveness of probabilistic modeling approaches in defect prediction ([Hosmer et al, 2005](#)).
- **NB (Naive Bayes)**: NB is a probabilistic model that applies Bayes' theorem with the assumption of independence between features. It is simple, computationally efficient, and often used in text classification tasks. By including NB, we aim to assess the performance of probabilistic classifiers in defect prediction scenarios ([Cestnik, 1990](#)).
- **RF (Random Forest)**: RF is an ensemble learning method that constructs multiple decision trees and combines their predictions to make the final classification. It is known for its robustness, ability to handle high-dimensional data, and resistance to overfitting. By including RF, we can investigate the benefits of ensemble techniques in defect prediction ([Breiman, 2001](#)).
- **SVM (Support Vector Machines)**: SVM is a powerful machine learning model that constructs hyperplanes to separate instances belonging to different classes. It aims to find the optimal decision boundary that maximally separates the classes. By incorporating SVM, we can evaluate the performance of margin-based classifiers in defect prediction ([Cortes and Vapnik, 1995](#)).
- **Voting Classifier**: The Voting Classifier is an ensemble model combining predictions from multiple individual models. It takes a majority vote (for classification) or averages the predicted probabilities (for probability estimation) to make the final

prediction. Using the VotingClassifier, we can assess the performance improvement achieved through ensemble techniques and determine if combining the predictions of multiple models leads to better defect prediction results (Dietterich, 2000).

The selection of these models was based on a thorough analysis of the literature, which revealed them or some variation of them to be the most frequently used algorithms for bug prediction. Also, the selection of these models provides a variety of approaches, such as decision tree-based models (CART), instance-based learning (KNN), linear discriminant-based models (LDA), probabilistic models (LR and NB), ensemble methods (RF and VotingClassifier), and margin-based classifiers (SVM). Through an evaluation of the model's performance in defect prediction, valuable insights can be obtained regarding their ability to effectively capture underlying patterns and accurately predict defects within the Apache Commons systems.

3.4 Training Phase

The training phase is a crucial step in building effective defect prediction models, and we conducted the training phase using the selected models on the Apache Commons dataset. The following steps were followed to train the models:

- **Data Preprocessing:** Before training the models, the dataset was preprocessed to ensure its suitability for machine learning. It involved checking for missing values, normalizing and scaling the numerical features, and encoding categorical features. To address the issue of class imbalance in our dataset, we employed the *Synthetic Minority Over-sampling Technique* (SMOTE). By applying SMOTE, we aimed to alleviate the class imbalance problem and ensure a more balanced representation of the majority and minority classes in our training data.
- **Cross-Validation:** A cross-validation strategy was employed during the training phase to ensure robustness and avoid overfitting. The dataset was separated into k-folds ($k = 10$), and each fold was used as a validation set while the remaining folds were employed for training. This process was repeated k times, with each fold serving as the validation set once. Cross-validation allows for a more reliable assessment of the model's performance by mitigating the impact of data partitioning on the results.
- **Model Training:** Each selected model, including CART, KNN, LDA, LR, NB, RF, SVM, and the VotingClassifier, was trained using the preprocessed dataset. The models were instantiated with their respective hyperparameters, pre-defined by the Python library `sciKitLearn`.
- **Model Evaluation:** After training, the performance of each model was evaluated using various evaluation metrics, including precision, recall, F1 score, accuracy, and the area under the ROC curve (AUC). These metrics comprehensively assess the model's ability to correctly classify defects and its overall predictive performance. The evaluation was conducted on the training data (to assess model overfitting) and the validation data (to assess generalization).

These steps ensured that the selected models were trained using a standardized and rigorous process. The combination of data preprocessing, cross-validation, and evaluation allowed for the development of accurate defect prediction models. The trained models were subsequently used for comparative analysis between single and ensemble models (VotingClassifier) to determine their effectiveness in defect prediction.

3.5 Ensemble Model Construction

An ensemble model was constructed using the VotingClassifier technique to compare ensemble and single-model performances. The VotingClassifier combines the predictions of multiple individual models to make the final prediction. This study employed the particular models' CART, KNN, LDA, LR, NB, RF, and SVM as base estimators for the ensemble model.

The ensemble model construction involved the following steps: (1) **Base Estimator Selection**: The base estimators for the ensemble model were selected based on their performance and diversity. CART, KNN, LDA, LR, NB, and SVM were deemed suitable candidates, offering various model types and characteristics. (2) **Voting Scheme**: The VotingClassifier supports different voting schemes, such as "hard" or "soft" voting. This study employed "soft" voting, considering each base estimator's predicted probabilities. The predicted probabilities were averaged to generate the final probability estimates for defect prediction. It allows the ensemble model to consider the confidence level of each base estimator's prediction. (3) **Ensemble Model Training**: The ensemble model was trained using the preprocessed dataset and the selected base estimators. The training process involved fitting the ensemble model to the training data, which trained each base estimator using the same data. The base estimators' predictions were then combined using the soft voting scheme to generate the ensemble model's final prediction. (4) **Ensemble Model Evaluation**: The performance of the ensemble model was evaluated using the same evaluation metrics used for the single models, including precision, recall, F1 score, accuracy, and AUC. The evaluation was conducted on the training data (to assess potential overfitting) and the validation data (to assess generalization).

We constructed the ensemble model using the VotingClassifier technique to assess whether combining the predictions of multiple models can improve defect prediction performance compared to individual models. We then compared the ensemble model's evaluation results with those of the single models to determine the effectiveness of the ensemble approach in defect prediction.

3.6 Performance Evaluation

We evaluated the defect prediction models' performance to assess their effectiveness in identifying and predicting software defects. The evaluation metrics used in this study included precision, recall, F1 score, accuracy, and area under the ROC curve (AUC). These metrics provide comprehensive insights into the models' performance from different perspectives. The evaluation was performed on both the single models and the ensemble model.

- **Precision:** Precision measures the proportion of correctly predicted defects out of all instances classified as defects. It indicates the model’s ability to avoid false positives or instances incorrectly identified as defects. Higher precision values indicate a lower rate of false positives, signifying more accurate predictions of actual defects.
- **Recall:** Recall, also known as sensitivity or true positive rate, measures the proportion of correctly predicted defects out of all actual defects. It indicates the model’s ability to identify all positive instances accurately without missing any defects. Higher recall values indicate a lower rate of false negatives, signifying better coverage of true defects.
- **F1 score:** The F1 score is the harmonic mean of precision and recall. It provides a balanced measure that considers precision and recall, overall assessing the model’s performance. The F1 score is useful when there is an imbalance between the number of defects and non-defect instances.
- **Accuracy:** Accuracy measures the overall correctness of the model’s predictions by calculating the proportion of correctly classified instances out of all instances. It indicates the model’s overall performance but can be misleading in class imbalance. Nevertheless, accuracy is widely used to assess the model’s general predictive capability.
- **Area under the ROC curve (AUC):** The AUC metric assesses the model’s ability to distinguish between defect and non-defect instances across different classification thresholds. It plots the true positive rate (recall) against the false positive rate, measuring the model’s overall discriminative power. Higher AUC values indicate better performance in separating defects from non-defects.

Each model’s performance evaluation metrics were calculated, including the single models (CART, KNN, LDA, LR, NB, RF, SVM) and the ensemble model (Voting-Classifier). The evaluation was conducted using the validation data to assess the models’ generalization capability and identify the best-performing approach for defect prediction.

By comparing the single and ensemble models’ performance across these metrics, we can determine which approach yields the most accurate, reliable, and well-calibrated predictions for software defects in the Apache Commons systems.

4 Results

In this section, we present and discuss the results obtained from our experiments on the efficacy of ensemble models for bug prediction. We analyze the performance of single and ensemble models across various evaluation metrics to gain insights into their comparative effectiveness. Additionally, we investigate the impact of different ensemble configurations on bug prediction accuracy and provide a detailed analysis of the observed results.

First, we report the performance metrics of single individual models, including CART, KNN, LDA, LR, NB, RF, and SVM. These metrics include accuracy, precision, recall, F1-score, and Area under the ROC curve (AUC). We analyze the strengths and limitations of each single model in capturing bug patterns and provide a comparative analysis of their performance.

Next, we present the performance results of the ensemble models constructed using the VotingClassifier approach. We evaluate the performance of ensemble models utilizing the same set of metrics as the single models and compare them against the individual models. This analysis allows us to determine how much ensemble models can improve bug prediction accuracy compared to single models. The resulting metrics of each model and subject system are shown in Table 2 and Table 3

Table 2 Results of each model for commons-compress

Model	Precision	Recall	F1	Accuracy	AUC
NB	0.9461	0.9430	0.9154	0.9430	0.5449
CART	0.9120	0.9330	0.9114	0.9330	0.6472
KNN	0.9082	0.9274	0.9165	0.9274	0.6123
LDA	0.9452	0.9430	0.9154	0.9430	0.6231
LR	0.9460	0.9430	0.9154	0.9430	0.6157
SVM	0.9450	0.9430	0.9154	0.9430	0.5320
RF	0.9128	0.9430	0.9169	0.9340	0.7068
EVC ¹	0.9462	0.9430	0.9153	0.9430	0.7217

¹EVC: Ensemble Voting Classifier

Table 3 Results of each model for commons-cli

Model	Precision	Recall	F1	Accuracy	AUC
NB	0.9645	0.8904	0.9185	0.8904	0.8618
CART	0.9733	0.9684	0.9696	0.9560	0.9202
KNN	0.9703	0.9555	0.9609	0.9555	0.9147
LDA	0.9418	0.9433	0.9347	0.9433	0.7918
LR	0.9544	0.9281	0.9196	0.9281	0.8398
SVM	0.9594	0.9576	0.9369	0.9576	0.6917
RF	0.9745	0.9684	0.9690	0.9684	0.9380
EVC ¹	0.9649	0.9626	0.9584	0.9626	0.9542

¹EVC: Ensemble Voting Classifier

4.1 RQ1: How does the performance of single and ensemble models compare for specific contexts?

To address the posed question, we evaluate the selected machine-learning models using a set of five metrics: (1) Precision, (2) Recall, (3) F1-score, (4) Accuracy, (5) AUC. These metrics help to ensure a thorough analysis of our results. Initially, we examine the outcomes yielded by each model in the context of each subject system. It allows an in-depth understanding of the model’s performance in isolated contexts. Subsequently, we expand our analysis to include a comparative discussion where we juxtapose the results obtained from both subject systems. This cross-project result analysis aims to draw meaningful insights and contrasts between single and ensemble models for

bug prediction. The comparative analysis is important to illustrate how the models' performance may vary across different datasets, contexts, and metrics.

The sections are organized as follows: first, we discuss each model and model type (single or ensemble) in terms of its metrics for the commons-compress subject system. After that, we analyze the results in the context of the commons-cli subject system and compare the results obtained in both systems.

4.1.1 Result Analysis for Commons-compress

In this section, we describe the results obtained from experimenting with the selected models, considering the performance of each metric in the context of the commons-compress subject system. Also, we compare the results from commons-compress with those obtained for commons-cli.

Naive Bayes (NB) Results

While the NB model cannot outperform the other models in recall distinctly, it achieves a remarkable precision rate of 0.9461. This precision rate not only equals that of the EVC model but also surpasses the precision of all other individual models (CART, KNN, LDA, LR, and SVM). This result suggests that the NB model's defect predictions are likely to be accurate.

Although competitive, the NB F1-score is somewhat lower than that of the Random Forest (RF) model, indicating a well-maintained balance between precision and recall with opportunity for improvement.

The resulting precision of 0.9461 indicates a high level of specificity in the model's predictions. However, precision should be distinct from overall accuracy. The NB model accuracy metric (0.9430) aligns closely with the other models and suggests its effectiveness in identifying defective and non-defective cases. However, it does not necessarily mean the NB outperforms the different models in all aspects of prediction.

The NB's results have a noticeable restriction in that it has the lowest Area Under Curve (AUC) score (0.5449) among practically all other models. Thus, although the NB showed exceptional precision and precision scores, this lower AUC indicates a reduced capability to differentiate between positive and negative classes effectively.

The observed discrepancy in the Naive Bayes (NB) results, exhibiting high accuracy and precision yet possessing a low Area Under Curve (AUC) score for the commons-compress dataset, might initially seem paradoxical. However, it can be clarified by delving into the subtleties of these evaluation metrics and the specific characteristics of the dataset in question.

Accuracy and precision are metrics that reflect the count of accurate predictions by the model. High accuracy indicates the model's proficiency in correctly identifying defective and non-defective cases. Precision, in contrast, specifically denotes the model's skill in accurately predicting defective (buggy) cases, suggesting a high likelihood of correctness when the model classifies a case as buggy.

On the other hand, the AUC measures the model’s capacity to differentiate between classes- in this scenario, between buggy and non-buggy classes. A lower AUC score indicates relative ineffectiveness in this differentiation task.

The divergence between precision, recall, and AUC in the NB results can be caused by several factors, with an imbalanced dataset being more likely, especially in a bug prediction context. In cases where non-defective instances significantly outnumber defective ones, the model may become adept at predicting the majority class, thereby boosting accuracy and precision. However, this does not inherently imply a robust ability to effectively distinguish between buggy and non-buggy classes.

Despite its limitations within the context of the commons-compress system, the Naive Bayes (NB) model demonstrates a good balance between precision and recall, as reflected in its F1 score of 0.9154.

While the Naive Bayes (NB) model is distinguished by its precision, its considerably lower Area Under Curve (AUC) score, compared to ensemble models and other single models, highlights potential limitations. Alternative models may be more effective for defect prediction, particularly in scenarios with substantial implications of misclassification. Consequently, it is wise to carefully assess the unique needs of the defect prediction task when deciding whether to employ the NB model or to investigate other available options.

CART (Classification and Regression Trees) Results

The CART model showed a lower precision (0.9120) than the NB and EVC models. It means it has a higher rate of false positives than the best model (EVC with a precision of 0.9462), labeling some non-defective cases as defects more frequently. It is worth noting that the recall of the CART is comparable to that of the other models for the commons-compress dataset (0.9330).

The higher rate of false positives that the CART can generate has a detrimental effect on the amount of time and resources required to investigate false positives, which are non-buggy classes mistakenly labeled as buggy (Yanyang et al, 2022). Its poor F1-score of 0.9114, which is exceeded by the RF (0.9169), provides additional evidence that this is the case.

The CART’s AUC serves as an additional demonstration of the model’s inability to distinguish between positive and negative classes. The results produced by the CART’s model demonstrate that it encounters difficulties in identifying meaningful patterns inside the commons-compress dataset compared to the other models examined. Additionally, because bug prediction datasets are typically imbalanced (Song et al, 2019), the CART model may be biased towards the majority class, resulting in poor performance.

KNN (K-Nearest Neighbors) Results

Although the KNN model's F1-score of 0.9165 suggests a reasonable balance between precision and recall for the commons-compress bug prediction task, a closer examination reveals significant limitations. Specifically, the model exhibits the **lowest precision (0.9082) among the compared models, indicating a tendency to generate false positives**. It means the KNN model is more likely to label code as buggy when bug-free incorrectly. Adopting KNN as a defect predictor for commons-compress could reduce the efficiency of the development process. Developers might be expending resources and attention on error-free parts of the code.

In addition to likely generating a substantial number of false positives, the KNN model also demonstrates the lowest recall (0.9274). This aspect is particularly concerning, as it suggests that the KNN model is prone to overlooking actual bugs, mistakenly categorizing code as non-buggy (false negatives). Such oversights are critical, as they could allow undetected bugs to slip through into production, potentially compromising the software's functionality and integrity.

The KNN model demonstrates a low score (0.6123) for the AUC metric, particularly compared to other models, notably the ensemble EVC. This implies that KNN needs to improve its ability to distinguish between buggy and non-buggy classes. As we discussed previously, this inability of KNN is also evident in the precision and recall metrics.

Based on the AUC score, the KNN model may not be optimal when high precision is important or when a distinct classification of classes is required. The KNN algorithm's strength lies in its simplicity, as it makes little assumptions about the dataset's structure. Nevertheless, this could also pose a constraint in intricate or high-dimensional environments where the notion of proximity may not be as easily grasped. Moreover, KNN can be susceptible to imbalanced data and noisy features frequently encountered in bug prediction applications. The poorer precision and AUC ratings for the commons-compress can be attributed to this sensitivity.

LDA (Linear Discriminant Analysis) Results

The LDA model demonstrates a high precision rate of 0.9452, which is close to the top precision rates achieved by the Naive Bayes (NB) and Ensemble Voting Classifier (EVC) models, with precision values of 0.9461 and 0.9462, respectively. Its recall and accuracy stand at 0.9430, aligning it with the performance of other models. While the LDA's F1-score is 0.9154, indicating a commendable balance between precision and recall, it is not the highest among the models.

The LDA's Area Under the Curve (AUC) score is 0.6231. Although not the lowest, this score is considerably inferior to the more sophisticated ensemble model, EVC (0.7217). **This lower AUC score implies that the LDA is comparatively less capable of distinguishing between defective and non-defective classes than an ensemble approach.**

When applied to the commons-compress subject system, the LDA model demonstrates robust precision and recall metrics performance. This high level of precision

and recall suggests that the LDA is dependable in accurately predicting defects and effectively identifying genuine cases of defects. However, its AUC score needs to be higher than the top-performing models. This lower AUC indicates that, although the LDA can predict defects reasonably, it could be more adept at ranking these predictions according to their likelihood of defect. This aspect is important for understanding the LDA’s effectiveness in defect prediction within the commons-compress context.

The LDA algorithm assumes that the classes in the dataset are normally distributed and that they have equal covariance (Iosifidis et al, 2013). If these assumptions are unknown, the effectiveness of the LDA model may be compromised. It could contribute to the relatively lower AUC score observed for LDA compared to other models. The AUC is a crucial measure reflecting the model’s capacity to rank predictions correctly. If the actual data distribution deviates significantly from LDA’s foundational assumptions, the model’s ability to accurately rank these predictions could be impaired. As a result, other models, such as the Ensemble Voting Classifier (EVC), might demonstrate more consistent and robust performance across various metrics, including the AUC. In scenarios where the underlying data characteristics do not align well with LDA’s assumptions, alternative models like EVC could be more suitable for ensuring higher overall predictive accuracy.

LR (Logistic Regression) Results

The accuracy of the LR model, 0.9430, demonstrates its high reliability in bug prediction. The remarkable precision showcased in our experiments emphasizes its expertise in detecting real errors and reflects its ability to identify bug-free code accurately.

The AUC for the LR model (0.6980) is quite strong, outperforming some other studied models, such as the SVM (0.5320) and the NB (0.5449). However, it doesn’t measure up to the EVS, which has an AUC of 0.7217. This analysis suggests that while the LR model reasonably discriminates between defective and non-defective classes, it might not be the most efficient model. Its precision measurements highlight this even further. Even though the LR’s precision is not the best, it is still reasonably good.

An excellent ability to discriminate between defective and non-defective classes becomes especially important when the main objective is to reduce false positives - a scenario where the absolute accuracy of defect prediction is crucial. Therefore, although generally reliable and effective, there may be better options than the LR model in situations where the utmost precision in predicting bugs is crucial.

SVM (Support Vector Machine) Results

The SVM model’s precision is 0.9309, indicating its resilience in accurately identifying faults while minimizing false positives. Although its precision is not the highest compared to the other models examined, it is nonetheless remarkable. Furthermore, the model demonstrates recall and accuracy similar to the different models, highlighting

its effectiveness in detecting real errors while preserving a balanced trade-off between precision and recall.

However, the SVM’s performance is overshadowed by its AUC metric, which is the lowest compared to the other models evaluated. This lower AUC score highlights a significant decrease in the SVM’s ability to effectively differentiate between defective and non-defective classes relative to the analyzed models.

SVM models are known for their proficiency in high-dimensional spaces and their capability to model non-linear relationships (Nayak et al, 2015). However, its success is significantly dependent on the precise adjustment of hyperparameters. The suboptimal AUC score observed in our tests (0.5320) may imply that the default hyperparameters are not ideally suited or that the SVM’s decision boundary needs to encapsulate the characteristics of the data effectively. This observation suggests the potential for further optimization through hyperparameter adjustment or reconsidering the model’s suitability for the dataset.

RF (Random Forest) Results

The Random Forest (RF) model exhibits a precision of 0.9344, marginally behind the highest precision models, Naive Bayes (NB) with 0.9462 and Ensemble Voting Classifier (EVC) with 0.9461. This high precision level indicates that the RF maintains a relatively low false-positive rate. Furthermore, the RF shows the highest F1 score among all the models, suggesting a good balance between precision and recall. Its AUC is also high (0.7068), although not the highest.

Regarding precision, recall, accuracy, and F1-score, the RF produced high scores, exhibiting robust performance across most metrics. It makes it a good candidate for bug prediction in the context of the commons-compress dataset. It effectively combines high accuracy with a strong balance between precision and recall, capturing this dataset’s nuances well.

The RF is an ensemble learning method, building its strength by constructing multiple decision trees. Its primary advantage lies in its capacity to mitigate overfitting while minimizing error due to bias. Generally, RF is well-suited for classification problems, particularly in complex datasets with non-linear relationships between features. However, its marginally lower AUC score than the EVC might indicate that the EVC approach could be slightly more adept at capturing subtle nuances in the data in the context of commons-compress. This observation could guide further investigation or model selection for optimal results in the context of bug prediction.

EVC (Ensemble Voting Classifier) Results

The EVC and NB models achieve the highest precision compared to all others. EVC demonstrates a weak indicator of false positives, greatly enhancing its reliability in

predicting defects. The model exhibits a competitive F1 score, which suggests a favorable equilibrium between precision and recall, albeit somewhat lower than the RF model, which has the highest-performing F1 score.

The EVC exhibits strong performance across multiple criteria, making it a reliable option for defect prediction in the commons-compress dataset. The model’s high AUC score is particularly notable, as this measure is essential for assessing the model’s effectiveness in distinguishing between classes.

In our experiments, the EVC aggregates the predictions of individual models (CART, KNN, LDA, LR, NB, and SVM), leading to enhanced performance, specifically in precision and AUC, as evident from the reported data. The ensemble technique leverages the advantages of individual models while minimizing their limitations, resulting in predictions that are often more consistent.

4.1.2 Result Analysis for Commons-cli

In this section, we expand the results analysis of each model considering the performance of each metric in the context of the commons-cli subject system. Also, we compare the results from commons-cli with those obtained for commons-compress.

Naive Bayes (NB) Results

In analyzing the commons-cli dataset, the Naive Bayes (NB) model achieved a precision of 0.9645. This precision rate is high, though lower than that exhibited by the ensemble Random Forest (RF) model.

When considering other key metrics such as recall, accuracy, and the F1 score, NB’s performance is the least impressive among all the models tested on the commons-cli dataset, recording values of 0.8904 for both recall and accuracy and 0.9185 for the F1 score. These scores indicate that the NB model may require improvements or adjustments to align with the dataset’s characteristics.

Regarding the AUC, the NB model shows an interesting trend. Its AUC in the commons-cli dataset surpasses its performance in the commons-compress dataset, suggesting a relative improvement or better suitability to the former. While its AUC is not the lowest among the evaluated models for the commons-cli dataset, it still does not outperform the EVC model, which holds the highest AUC score. This comparison underscores the EVC’s superior ability to discriminate between classes within this dataset, a crucial aspect for consideration in predictive modeling.

Overall, the NB model demonstrates good precision in the commons-cli dataset, indicating a high likelihood of accuracy when it predicts a defect. However, the most notable limitation of the NB model in this context is its recall capability. It fails to identify a significant number of actual defects. Additionally, its accuracy is lower than other models, which could cause concern.

Despite achieving a satisfactory AUC score, the combination of low recall and accuracy suggests that NB may not be the most effective single model for defect prediction in the commons-cli dataset. The performance of the ensemble models considered notably surpasses it. When analyzing the commons-cli and commons-compress

datasets, the NB model consistently shows high precision but struggles with recall. This pattern suggests the NB model tends to be conservative in predicting defects, opting for a higher confidence level in its positive predictions. However, this approach has the downside of overlooking several actual bugs.

In the case of AUC, NB performs better in the commons-cli dataset than in the commons-compress dataset. However, it still does not achieve the highest score, indicating that while NB has a reasonable capability in ranking predictions, other models might provide superior overall performance. The decision to employ NB should be informed by the specific trade-offs deemed acceptable for a given defect prediction task. Particular consideration should be delivered to scenarios where high precision is prioritized over recall. Nonetheless, for a more balanced approach that considers all performance metrics, ensemble methods or other high-performing models might be more suitable than NB.

CART (Classification and Regression Trees) Results

In the commons-cli dataset, the CART model has a precision of 0.9733, slightly lower than the top-performing RF model's precision of 0.9745. The CART model's precision score demonstrates that it produces accurate defect predictions. Furthermore, CART exhibits great accuracy, showing its capacity to correctly classify occurrences across both defect and non-defect classes.

However, whereas CART has a solid AUC of 0.9202, it is lower than the EVC's AUC of 0.9542. It suggests that, while CART is good at distinguishing between classes, the EVC performs better.

A comparison of CART's performance on the commons-compress and commons-cli datasets reveals significant differences. While CART's performance on the former is good, it is low in parameters like precision and AUC. It is important to note that CART's performance improves significantly on the latter.

These performance variations are most likely due to differences between the two datasets. Data distribution and how well the CART algorithm captures the underlying patterns in each dataset can influence the model's result. These observed variances highlight the relevance of considering dataset-specific details when evaluating the applicability and efficacy of prediction models such as CART in various contexts.

KNN (K-Nearest Neighbors) Results

The KNN model demonstrates high accuracy for the commons-cli dataset, with a value close to the best precision the RF model achieves (0.9745). It indicates that the KNN model is quite assertive in its predictions of defects. The model exhibits a recall score of 0.9555, which means a substantial improvement compared to the results of the commons-compress dataset. The KNN model has a high F1-score of 0.9609, suggesting a good balance between precision and recall. The accuracy of the KNN model is

significantly high and comparable to other models when evaluated on the commons-cli dataset. The AUC in the commons-cli dataset is higher than commons-compress, but it is not the highest (EVC - 0.9542).

Upon evaluating the performance of the KNN model on both datasets, commons-cli, and commons-compress, one may observe that the KNN model exhibits significantly superior performance in the commons-cli dataset compared to the commons-compress dataset. The improvement in the AUC score indicates that KNN is more proficient in distinguishing between defective and non-defective instances in the commons-cli dataset. This discrepancy may arise for several reasons, including disparities in the feature space, data distribution, or the structural arrangement of data points' neighborhoods in each dataset.

The disparity in KNN's performance across the two studied datasets underscores the significance of the data attributes in model selection for defect prediction. Although KNN may not excel in any specific metric, its precision and recall scores are compelling for defect prediction, particularly in the commons-cli dataset. Nevertheless, while dealing with the commons-compress dataset, it is worth considering other models that provide superior precision and enhanced discriminative capability, as evaluated by the AUC metric.

LDA (Linear Discriminant Analysis) Results

The precision of the LDA model on the commons-cli dataset is the lowest (0.9418) among the other models in our experiments; however, it is still relatively high. This indicates that LDA has a higher rate of false positives for the commons-cli. Its recall is robust, suggesting its efficacy in detecting true faults. Although the F1-score indicates a favorable balance between precision and recall, there is room for improvement despite the low precision and high recall.

The LDA's AUC score is superior in the commons-cli dataset compared to commons-compress. However, it still needs to achieve the performance level of the ensemble model EVC. Across both datasets, the LDA exhibits robust recall and reasonable accuracy. However, its precision, F1-score, and AUC do not regularly reach high levels. Compared to the commons-compress dataset, the model's performance on the commons-cli dataset demonstrates improvements, particularly in ACU, compared to the commons-compress dataset. The observed different scores depending on the dataset suggest that the specific attributes of the dataset heavily influence the efficacy of LDA.

Although LDA is a robust model for defect prediction, particularly when high recall is prioritized, it may not be the optimal choice for applications that prioritize the highest precision and discriminative ability, as indicated by its low AUC. When choosing between LDA and other defect prediction models, it is important to consider the specific requirements and preferences of the prediction task.

LR (Linear Regression) Results

The LR model demonstrates a good precision score in the commons-cli dataset, suggesting a relatively low occurrence of false positives in the results. However, there are better models in terms of precision when compared to the other models in the study.

The LR model achieved a high AUC in the commons-cli dataset, indicating its great discriminatory power in distinguishing between buggy and non-buggy classes. However, its performance is lower than that of the best model (EVC, with an AUC of 0.9542).

Comparing the performance of the LR model across the commons-compress and commons-cli datasets reveals it is highly effective. Particularly in the commons-cli dataset, LR shows good metric scores, especially in terms of AUC. The high AUC score suggests that LR is well-suited for defect prediction tasks within the context of the commons-cli.

The LR model's performance across the studied datasets underscores its versatility as a binary classifier. Although it may not consistently rank as the top performer in every metric, its overall well-rounded performance makes it a reliable option for defect prediction, particularly when a good balance between several evaluation metrics is required. The specific goals and limitations of the defect prediction task, such as the relative significance of precision versus recall or the requirement for discriminative capability, should determine the choice between LR and other models.

SVM (Support Vector Machine) Results

The SVM model's precision score is marginally below that achieved by the RF model. Although its recall is high, it doesn't match the scores obtained by CART or RF.

The SVM's F1 score is high, reflecting a good balance between precision and recall. Additionally, its accuracy is expressive, indicating its capability to classify a significant portion of instances within the dataset correctly.

Despite its robustness in precision and accuracy, SVM's lower AUC in both the commons-cli and commons-compress datasets implies potential limitations for scenarios where the ranking or prioritization of defects based on likelihood is a key concern.

Across both datasets, SVM consistently exhibits high precision. Nevertheless, the comparatively lower AUC in commons-cli and commons-compress contexts indicates a challenge in effectively ranking predictions. When opting for SVM in defect prediction tasks, the decision should be informed by specific requirements, especially where high precision is essential but with less focus on discriminative ability, as indicated by the AUC. Alternative models like RF or EVC may be more suitable when a high AUC is critical.

RF (Random Forest) Results

The RF demonstrates high precision in the commons-cli dataset, indicating its defect predictions are reliable with minimal false positives. In addition, the RF and CART achieve the highest recall rate, accurately detecting most real defects.

The F1-score for the RF model is also high, almost on par with CART, suggesting an outstanding balance between precision and recall. The model achieves the highest accuracy compared to the EVC, effectively classifying defects and non-defects.

When analyzing the performance of RF on the commons-compress and commons-cli datasets, it becomes clear that RF is a reliable and adaptable model for defect prediction. In both datasets, the performance of RF is outstanding, with awe-inspiring results in the commons-cli dataset. The consistent performance across different datasets highlights the versatility of RF across different datasets.

The ensemble nature of the RF, with its combination of multiple decision trees, is likely a key factor in its high scores. **This approach helps mitigate the risk of overfitting and allows for capturing intricate patterns within the data.** The AUC in the commons-cli dataset further demonstrates its effectiveness in ranking predictions, a crucial aspect in defect prediction tasks where prioritization is crucial.

EVC (Ensemble Voting Classifier) Results

It is possible that the EVC can reduce the occurrence of false positives. This is a result of the accuracy demonstrated in the commons-cli dataset. The EVC recall is robust but does not rank among the top scores in the experiments. The effectiveness of the EVC in identifying real defects is comparable to CART and RF.

The F1-score for the EVC is high, suggesting a strong balance between precision and recall, although CART and RF outperform it slightly in this metric. The EVC demonstrates good accuracy compared to RF in the commons-cli dataset, which shows its effectiveness in accurately classifying defects and non-defects.

Regarding AUC, the EVC model stands out with the highest score among all the models studied in the commons-cli dataset. This shows its superior ability to distinguish between defective and non-defective classes.

When evaluating the performance of EVC on both the commons-compress and commons-cli datasets, it becomes evident that EVC is an efficient model for predicting defects. The EVC ensemble approach, which combines predictions from multiple models, is likely to enhance its performance by capitalizing on the strengths of each model and minimizing their limitations, as in the RF model.

The EVC's AUC is consistently high in both datasets, highlighting its effectiveness when accurate class differentiation is needed. The choice of the EVC model would be especially indicated when a good balance of accuracy and the capability to order and prioritize predictions accurately is necessary. The versatility and reliability of EVC make it a good choice for defect prediction.

4.2 RQ2: Does the ensemble model outperform the single models regarding defect prediction accuracy, precision, recall, F1 score, and AUC?

For the commons-compress dataset, all models showed similar accuracy and recall. Thus, the ensemble models do not outperform any studied single models regarding these metrics. The ensemble EVC and the single model NB have the highest precision, while other single models have lower precision. So, regarding precision, we have a tie between EVC and the single model NB. However, all the different single models were within the EVC's precision.

The RF ensemble has the highest F1 score, with another ensemble's EVC slightly lower. These results show that ensemble models perform better than single models when using the F1-score metric. The EVC and RF models outperform all single models.

For the commons-cli dataset, the EVC and RF share the highest accuracy and precision, once more outperforming all other single models. However, CART has the highest recall, while EVC has a lower recall. Here, EVC is outperformed by a single model. On the other hand, RF showed a high recall.

Although CART has the highest F1-score, with EVC slightly lower, the EVC and RF share the highest AUC, significantly outperforming the single models.

In a comparative analysis, EVC is consistently among the top performers for accuracy in both datasets, sharing the highest scores with RF in the commons-cli dataset. However, it doesn't outperform the metrics of some single model in the commons-compress dataset.

It is important to observe that while the precision of EVC and RF are the top performers in the experiments, the EVC does not lead to recall in either dataset. Regarding recall, single models like CART and RF showed higher scores.

The EVC, based on the predictions of CART, KNN, LDA, LR, NB, and SVM, with LR as the base model, shows robust performance across various metrics. While it doesn't consistently outperform all single models in every metric, it stands out regarding AUC, a critical measure for classification tasks, especially in contexts with imbalanced data, as bug prediction.

The EVC's strength lies in its balanced performance, particularly its discriminative ability, as evidenced by its AUC scores. It makes it a reliable choice for defect prediction tasks, especially when a balance across different performance metrics is desired. However, certain single models may occasionally be preferable for specific needs, such as the highest recall or precision. The specific requirements and constraints of the defect prediction task should guide the choice between EVC and single models.

4.3 RQ3: What are the trade-offs between single and ensemble models for defect prediction?

In the context of defect prediction, using single models or ensemble models like the EVC or the RF involves several trade-offs. These trade-offs can impact the model's effectiveness and applicability in various scenarios.

Single models, like the NB or LR, are more straightforward to interpret. It can be advantageous in scenarios where understanding the basis of a prediction is as important

as the prediction itself. Furthermore, single models can excel in precision, recall, and AUC. For example, CART may have the highest recall in a particular dataset, as the results show. If a specific metric has to be prioritized due to the nature of the bug prediction task, a single model excelling in one of the previously mentioned metrics may be preferred.

Single models are also generally more computationally efficient than ensemble models. They require less processing and memory, making them suitable for bug prediction activities with limited computational resources. However, they are more prone to overfitting or underfitting, especially if not correctly tuned or if the dataset has high variability or complexity as bug prediction datasets.

Ensemble models, on the other hand, often provide better accuracy and generalization than single models. By aggregating the predictions of multiple models, they reduce the risk of overfitting and can capture more complex patterns. Also, ensemble models can offer a more balanced performance across the metrics: precision, recall, F1-score, accuracy, and AUC. It is beneficial in defect prediction, where a trade-off between these metrics often exists.

As ensemble models combine the results of multiple models, they generally require more computational resources in processing and memory. They are also more complex than single models, and their results can be harder to interpret. This complexity can make understanding the specific reasons behind a prediction challenging.

Ensemble models are usually more robust to changes and variability in data, making them well-suited for defect prediction in diverse datasets from different software systems, where data can deviate from ideal or expected distributions.

The choice between using single or ensemble models for defect prediction hinges on the ultimate objective of detecting the bugs. If the goal is to maximize overall accuracy and balance across several performance metrics and if computational resources are not a significant constraint, ensemble models like EVC or RF are often preferable. However, if interpretability, computational efficiency, or excellence in a specific metric (like precision or recall) is more critical, a single model can be more appropriate.

The nature of the software system under analysis and the specific context of the defect prediction – such as the cost of false positives vs. false negatives – also play crucial roles in determining the best model to choose.

5 Discussion

In this study, we performed bug prediction experiments on commons-cli and commons-compress datasets. We evaluated the performance of several machine learning models, including decision tree-based models like CART and ensemble approaches like the Voting Classifier, in predicting bugs. Here, we discuss the results obtained from both datasets and draw insights from the models' performance.

Our analysis of the commons-cli dataset revealed exciting findings. **The RF model achieved the highest AUC score (0.9542), sharing the top spot with another ensemble model, the Voting Classifier.** It indicates ensemble models' strong discriminatory power and effectiveness in bug prediction. The RF model's performance was closely followed by that of the CART model, which also achieved a high AUC score of 0.9202. These

results suggest ensemble models, mainly RF, capture relevant patterns and features for bug prediction in the commons-cli dataset.

The Voting Classifier matched the top performance of the RF model with an AUC score of 0.9542, surpassing or equalling the AUC scores of all individual models. It indicates that combining the predictions of multiple models, including decision tree-based models, effectively enhances the ensemble’s discriminatory power. The Voting Classifier successfully leverages constituent models’ diverse perspectives and strengths to make more accurate bug predictions.

Turning our attention to the commons-compress dataset, we observed a different trend. Here, the EVC (Voting Classifier) outperformed all individual models with the highest AUC score of 0.7217, indicating its effectiveness in handling this dataset’s complexity and unique characteristics. This performance underscores the capability of ensemble approaches to combine various models’ predictions, overcoming the limitations of individual models and achieving better bug prediction accuracy.

Comparing the results between the two datasets, we observed variations in the performance of the models. While the RF model consistently performed well in both datasets, the effectiveness of other models varied. **The performance of the Voting Classifier was also dataset-dependent**, as it outperformed individual models in both commons-cli and commons-compress, with a particularly significant improvement in the latter.

Our findings highlight the importance of considering different approaches in bug prediction tasks. The ensemble approach can effectively leverage the strengths of individual models and compensate for their weaknesses, leading to improved bug prediction performance. **However, to maximize performance, the choice of models should be based on the dataset’s specific characteristics.**

Future research could explore additional models and ensemble techniques to enhance bug prediction accuracy. Investigating the impact of feature engineering and dataset-specific preprocessing techniques on model performance could provide valuable insights for future bug prediction studies.

Our study demonstrates the effectiveness of ensemble approaches, such as the RF and Voting Classifier, in bug prediction tasks. By combining the predictions of multiple models, these ensembles can overcome individual limitations and improve the accuracy of bug predictions. However, their performance can vary depending on the dataset, highlighting the need for careful model selection and customization. Our findings contribute to the growing body of research on bug prediction and provide practical insights for software developers and quality assurance teams in identifying and addressing software bugs.

6 Threats to Validity

While our study aimed to provide insights into bug prediction using machine learning models and ensemble approaches, it is important to acknowledge the potential threats to the validity of our findings. These threats include:

- **Sample Selection Bias:** The selection of datasets may introduce bias, as we focused on the "commons-cli" and "commons-compress" projects. These projects may not

represent the characteristics of all software projects, limiting the generalizability of our results. Future research can expand the study to include various software projects from different domains and sizes to mitigate sample selection bias. It would provide a more representative sample and enhance the generalizability of the results.

- **Feature Selection Bias:** The choice and engineering of features can significantly impact model performance. We employed a set of features based on existing literature and domain knowledge, but there may be other relevant features that should have been considered. It could affect the effectiveness of the models and ensemble approach.
- **Model Selection Bias:** Our study included a specific set of machine learning models, namely Bayes, Cart, KNN, LDA, LR, SVM, RF, and the Voting Classifier. While these models are commonly used in bug prediction studies, other models could yield different results. Our findings are limited to the models selected for evaluation. Future studies should consider including a broader range of machine learning models to mitigate model selection bias. It would allow for a more comprehensive assessment and comparison of different algorithms, reducing the potential bias associated with a limited set of models.
- **Dataset Size:** The size of the datasets used for bug prediction can influence the performance of the models. The "commons-cli" and "commons-compress" datasets may have varied sizes and characteristics, affecting the results' reliability and generalizability. Increasing the size of the datasets used for bug prediction can help mitigate the impact of dataset size. Collecting more data from multiple projects or considering open-source repositories with larger datasets would provide a more robust foundation for bug prediction analysis.
- **Dataset Imbalance:** Imbalanced datasets, where the number of bug instances is significantly lower than non-bug instances, can impact model performance. To address dataset imbalance, various techniques, such as oversampling minority classes, under-sampling majority classes, or employing ensemble methods designed explicitly for imbalanced datasets, can be applied. These techniques can help mitigate the impact of class imbalance and improve the performance of bug prediction models.
- **Overfitting and Generalization:** The models may have the potential to overfit the training data, resulting in overly optimistic performance on the evaluation data. To mitigate this threat, we employed cross-validation techniques and evaluated the models on independent test sets. However, it is necessary to consider the risk of overfitting and the generalization of the findings. Regularization, early stopping, and model validation on independent test sets should be employed to mitigate the risk of overfitting and improve generalization. Additionally, cross-validation approaches can provide more reliable estimates of model performance and reduce the risk of overfitting.

7 Future Work

While this study provides valuable insights into the efficacy of ensemble models for bug prediction, several avenues for future research could enhance our understanding and contribute to advancing this field. We highlight some potential directions for future work:

Exploration of Different Ensemble Techniques: We employed the VotingClassifier as our ensemble learning technique in this study. However, several other ensemble methods, such as stacking, boosting, and random subspaces, could be explored. Investigating the effectiveness of these alternative ensemble techniques in bug prediction could provide valuable insights into their comparative performance and potential benefits.

Incorporating Different Feature Selection Methods: Feature selection plays a crucial role in model performance. Future work could investigate the impact of different feature selection methods on the performance of ensemble models for bug prediction. Comparing the performance of ensemble models with various feature selection techniques could help identify the most effective combinations.

Investigation of Hybrid Approaches: Hybrid approaches that combine ensemble models with other techniques, such as active learning or semi-supervised learning, hold promise in bug prediction. Future research could explore integrating ensemble models with active learning strategies to intelligently select informative instances for labeling and improve the performance of bug prediction models.

Analysis of Scalability and Efficiency: The scalability and efficiency of ensemble models are important considerations, mainly when dealing with large-scale software systems. Future work could evaluate the scalability and efficiency of ensemble models for bug prediction on large and complex software projects. It could involve exploring parallelization techniques, distributed computing frameworks, or model compression methods to enhance the scalability and efficiency of ensemble models.

Interpretability and Explainability: Ensemble models, especially those combining multiple complex models, sometimes need more interpretability and explainability. Future research could address this challenge by developing techniques to provide insights into the decision-making process of ensemble models. It could involve analyzing the contribution of individual base models, identifying important features, or generating explanations for bug predictions made by the ensemble.

By addressing these areas of future research, we can further advance the understanding and practical applications of ensemble models for bug prediction. These avenues open up exciting opportunities for improving bug detection accuracy, model interpretability, and the overall reliability of bug prediction systems.

It is important to note that while this section provides suggestions for future work, it is a partial list. Other relevant areas may be explored based on the specific goals and context of the research.

8 Conclusion

In this study, we analyzed bug prediction for two software projects, commons-cli, and commons-compress, using various machine learning models and two approaches: single and ensemble. Our findings provide insights into these models' performance and the effectiveness of the ensemble technique in bug prediction tasks.

Our analysis revealed that the studied models exhibited different performance levels across the datasets. In the common-cli dataset, the RF model stood out, achieving the highest AUC score, which it shared with the Voting Classifier. It demonstrated its

strong discriminatory power and accuracy in bug prediction. The ensemble approach, particularly the Voting Classifier, effectively leverages the strengths of the individual models.

In contrast, individual models like CART performed well for the commons-compress dataset but did not achieve exceptionally high AUC scores. The Voting Classifier, however, significantly improved the bug prediction accuracy, reaching the highest AUC score among all models. It underscores the effectiveness of ensemble techniques in handling the complexities and unique characteristics of specific datasets.

Our study highlights the value of considering a range of models and ensemble approaches in bug prediction tasks. Ensemble approaches can effectively combine the strengths of various models, enhancing overall performance and mitigating individual weaknesses. It is particularly evident in our results, where the ensemble models consistently performed well across both datasets.

We recommend further research to explore a broader range of models and ensemble techniques enhancing bug prediction metrics. Investigating the impact of feature engineering and dataset-specific pre-processing techniques will yield additional insights, potentially improving model performance. Additionally, evaluating these bug prediction models' practical applicability in real-world software development scenarios would be a valuable contribution to the field.

References

- Ajit A, Acharya K, Samanta A (2020) A review of convolutional neural networks. In: 2020 International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE), pp 1–5, <https://doi.org/10.1109/ic-ETITE47903.2020.049>
- Alazba A, Aljamaan H (2022) Software defect prediction using stacking generalization of optimized tree-based ensembles. *Applied Sciences* 12(9). <https://doi.org/10.3390/app12094577>, URL <https://www.mdpi.com/2076-3417/12/9/4577>
- Aleithan R (2021) Explainable just-in-time bug prediction: Are we there yet? In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), pp 129–131, <https://doi.org/10.1109/ICSE-Companion52605.2021.00056>
- Ali AR, Ur Rehman A, Nawaz A, et al (2022) An ensemble model for software defect prediction. In: 2022 2nd International Conference on Digital Futures and Transformative Technologies (ICoDT2), pp 1–5, <https://doi.org/10.1109/ICoDT255437.2022.9787439>
- Arora I, Tatarwal V, Saha A (2015) Open issues in software defect prediction. *Procedia Computer Science* 46:906–912. <https://doi.org/https://doi.org/10.1016/j.procs.2015.02.161>, URL <https://www.sciencedirect.com/science/article/pii/S1877050915002252>, proceedings of the International Conference on Information and Communication Technologies, ICICT 2014, 3-5 December 2014 at Bolgatty Palace Island Resort, Kochi, India

- Bhandari K, Kumar K, Sangal A (2022) Data quality issues in software fault prediction: a systematic literature review. *Artificial Intelligence Review* pp 1–70. <https://doi.org/10.1007/s10462-022-10371-6>
- Bommi NS, Negi A (2023) A standard baseline for software defect prediction: Using machine learning and explainable ai. In: 2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC), pp 1798–1803, <https://doi.org/10.1109/COMPSAC57700.2023.00278>
- Borg M, Svensson O, Berg K, et al (2019) Szz unleashed: An open implementation of the szz algorithm - featuring example usage in a study of just-in-time bug prediction for the jenkins project. Association for Computing Machinery, New York, NY, USA, MaLTesQuE 2019, pp 7–12, <https://doi.org/10.1145/3340482.3342742>, URL <https://doi.org/10.1145/3340482.3342742>
- Breiman L (2001) Random forests. *Machine Learning* 45(1):5–32
- Breiman L, Friedman JH, Olshen RA, et al (1984) *Classification and Regression Trees*. CRC press
- Caruana R, Niculescu-Mizil A (2006) An empirical comparison of supervised learning algorithms. In: *Proceedings of the 23rd International Conference on Machine Learning*. Association for Computing Machinery, New York, NY, USA, ICML '06, p 161–168, <https://doi.org/10.1145/1143844.1143865>, URL <https://doi.org/10.1145/1143844.1143865>
- Cestnik B (1990) Estimating probabilities: A crucial task in machine learning. In: *Proceedings of the 9th European Conference on Artificial Intelligence*. Pitman Publishing, Inc., USA, ECAI'90, pp 147–149
- Chung J, Teo J (2023) Single classifier vs. ensemble machine learning approaches for mental health prediction. *Brain Informatics* 10(1):1
- Cortes C, Vapnik V (1995) Support-vector networks. *Journal of Mach Learn* 20(3):273–297. <https://doi.org/10.1023/A:1022627411411>, URL <https://doi.org/10.1023/A:1022627411411>
- Cover T, Hart P (1967) Nearest neighbor pattern classification. *IEEE Transactions on Information Theory* 13(1):21–27. <https://doi.org/10.1109/TIT.1967.1053964>
- Dejaeger K, Verbraken T, Baesens B (2013) Toward comprehensible software fault prediction models using bayesian network classifiers. *IEEE Transactions on Software Engineering* 39(2):237–257. <https://doi.org/10.1109/TSE.2012.20>
- Dietterich TG (2000) Ensemble methods in machine learning. In: *International Workshop on Multiple Classifier Systems*

- Fernández-Delgado M, Cernadas E, Barro S, et al (2014) Do we need hundreds of classifiers to solve real world classification problems? *J Mach Learn Res* 15(1):3133–3181
- Fisher RA (1936) The use of multiple measurements in taxonomic problems. *Annals of Eugenics* 7(2):179–188
- Habibi PA, Amrizal V, Bahaweres RB (2018) Cross-project defect prediction for web application using naive bayes (case study: Petstore web application). In: 2018 International Workshop on Big Data and Information Security (IWBIS), pp 13–18, <https://doi.org/10.1109/IWBIS.2018.8471701>
- Hosmer DW, Lemeshow S, Sturdivant RX (2005) Introduction to the logistic regression model
- Iosifidis A, Tefas A, Pitas I (2013) On the optimal class representation in linear discriminant analysis. *IEEE Transactions on Neural Networks and Learning Systems* 24(9):1491–1497. <https://doi.org/10.1109/TNNLS.2013.2258937>
- Jacob RJ, Kamat RJ, Sahithya NM, et al (2021) Voting based ensemble classification for software defect prediction. In: 2021 IEEE Mysore Sub Section International Conference (MysuruCon), pp 358–365, <https://doi.org/10.1109/MysuruCon52639.2021.9641713>
- Jahanshahi H, Cevik M, Başar A (2020) Predicting the number of reported bugs in a software repository. In: Goutte C, Zhu X (eds) *Advances in Artificial Intelligence*. Springer International Publishing, Cham, pp 309–320
- Johnson F, Oluwatobi O, Folorunso O, et al (2022) Optimized ensemble machine learning model for software bugs prediction. *Innov Syst Softw Eng* 19(1):91–101. <https://doi.org/10.1007/s11334-022-00506-x>, URL <https://doi.org/10.1007/s11334-022-00506-x>
- Kamei Y, Shihab E (2016) Defect prediction: Accomplishments and future challenges. In: 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER), pp 33–45, <https://doi.org/10.1109/SANER.2016.56>
- Luo J, Luo R (2023) Research on image recognition based on reinforcement learning. In: 2023 4th International Conference on Computer Vision, Image and Deep Learning (CVIDL), pp 25–28, <https://doi.org/10.1109/CVIDL58838.2023.10166036>
- Ma W, Chen L, Zhang X, et al (2020) Impact analysis of cross-project bugs on software ecosystems. Association for Computing Machinery, New York, NY, USA, ICSE ’20, p 100–111, <https://doi.org/10.1145/3377811.3380442>, URL <https://doi.org/10.1145/3377811.3380442>

- Martinez ZH, Moreno PM, Vergara Camacho JA, et al (2023) Study on the use of defect metrics in the software development process. flaws and vulnerabilities. In: 2023 IEEE International Conference on Engineering Veracruz (ICEV), pp 1–7, <https://doi.org/10.1109/ICEV59168.2023.10329656>
- Maulud D, Mohsin Abdulazeez A (2020) A review on linear regression comprehensive in machine learning. *Journal of Applied Science and Technology Trends* 1:140–147. <https://doi.org/10.38094/jastt1457>
- Nagappan N, Ball T (2005) Use of relative code churn measures to predict system defect density. In: *Proceedings. 27th International Conference on Software Engineering, 2005. ICSE 2005.*, pp 284–292, <https://doi.org/10.1109/ICSE.2005.1553571>
- Nayak J, Naik B, Behera PDH (2015) A comprehensive survey on support vector machine in data mining tasks: Applications & challenges. *International Journal of Database Theory and Application* 8:169–186. <https://doi.org/10.14257/ijdta.2015.8.1.18>
- Nisbet R, Miner G, Yale K (2017) *Handbook of Statistical Analysis and Data Mining Applications*, Second Edition, 2nd edn. Academic Press, Inc., USA
- Okutan A, Yıldız OT (2014) Software defect prediction using bayesian networks. *Empirical Softw Engg* 19(1):154–181. <https://doi.org/10.1007/s10664-012-9218-8>, URL <https://doi.org/10.1007/s10664-012-9218-8>
- Pachouly J, Ahirrao S, Kotecha K, et al (2022) A systematic literature review on software defect prediction using artificial intelligence: Datasets, data validation methods, approaches, and tools. *Engineering Applications of Artificial Intelligence* 111:104773. <https://doi.org/https://doi.org/10.1016/j.engappai.2022.104773>, URL <https://www.sciencedirect.com/science/article/pii/S0952197622000616>
- Qu Y, Chi J, Yin H (2021) Leveraging developer information for efficient effort-aware bug prediction. *Information and Software Technology* 137:106605. <https://doi.org/10.1016/j.infsof.2021.106605>, URL <https://doi.org/10.1016/j.infsof.2021.106605>
- Rahim A, Hayat Z, Abbas M, et al (2021) Software defect prediction with naïve bayes classifier. In: *2021 International Bhurban Conference on Applied Sciences and Technologies (IBCAST)*, pp 293–297, <https://doi.org/10.1109/IBCAST51254.2021.9393250>
- Rathore SS, Kumar S (2017) Linear and non-linear heterogeneous ensemble methods to predict the number of faults in software systems. *Knowledge-Based Systems* 119:232–256. <https://doi.org/https://doi.org/10.1016/j.knosys.2016.12.017>, URL <https://www.sciencedirect.com/science/article/pii/S0950705116305202>

- Saharudin SN, Wei KT, Na KS (2020) Machine learning techniques for software bug prediction: A systematic review. *Journal of Computer Science* 16(11):1558–1569
- Shan C, Chen B, Hu C, et al (2014) Software defect prediction model based on lle and svm. In: 2014 Communications Security Conference (CSC 2014), pp 1–5, <https://doi.org/10.1049/cp.2014.0749>
- Shen Y, Hu S, Cai S, et al (2022) Software defect prediction based on bayesian optimization random forest. In: 2022 9th International Conference on Dependable Systems and Their Applications (DSA), pp 1012–1013, <https://doi.org/10.1109/DSA56465.2022.00149>
- Soe YN, Santosa PI, Hartanto R (2018) Software defect prediction using random forest algorithm. In: 2018 12th South East Asian Technical University Consortium (SEATUC), pp 1–5, <https://doi.org/10.1109/SEATUC.2018.8788881>
- Sofian H, Yunus NAM, Ahmad R (2022) Systematic mapping: Artificial intelligence techniques in software engineering. *IEEE Access* 10:51021–51040. <https://doi.org/10.1109/ACCESS.2022.3174115>
- Song Q, Guo Y, Shepperd M (2019) A comprehensive investigation of the role of imbalanced learning for software defect prediction. *IEEE Transactions on Software Engineering* 45(12):1253–1269. <https://doi.org/10.1109/TSE.2018.2836442>
- Tantithamthavorn C, Hassan AE (2018) An experience report on defect modelling in practice: Pitfalls and challenges. In: Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice. Association for Computing Machinery, New York, NY, USA, ICSE-SEIP '18, pp 286–295, <https://doi.org/10.1145/3183519.3183547>, URL <https://doi.org/10.1145/3183519.3183547>
- Tantithamthavorn CK, Jiarpakdee J (2021) Explainable ai for software engineering. In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp 1–2, <https://doi.org/10.1109/ASE51524.2021.9678580>
- Tosun A, Turhan B, Bener A (2008) Ensemble of software defect predictors: A case study. In: Proceedings of the Second ACM-IEEE International Symposium on Empirical Software Engineering and Measurement. Association for Computing Machinery, New York, NY, USA, ESEM '08, pp 318–320, <https://doi.org/10.1145/1414004.1414066>, URL <https://doi.org/10.1145/1414004.1414066>
- Turhan B, Bener A (2009) Analysis of naive bayes' assumptions on software fault data: An empirical study. *Data Knowl Eng* 68(2):278–290. <https://doi.org/10.1016/j.datak.2008.10.005>, URL <https://doi.org/10.1016/j.datak.2008.10.005>
- Varshitha KS, Kumari CG, Hasvitha M, et al (2023) Natural language processing using convolutional neural network. In: 2023 7th International Conference on Computing Methodologies and Communication (ICCMC), pp 362–367, <https://doi.org/>

[10.1109/ICCMC56507.2023.10083608](https://doi.org/10.1109/ICCMC56507.2023.10083608)

- Wangoo DP (2018) Artificial intelligence techniques in software engineering for automated software reuse and design. In: 2018 4th International Conference on Computing Communication and Automation (ICCCA), pp 1–4, <https://doi.org/10.1109/CCAA.2018.8777584>
- Wolpert D, Macready W (1997) No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation* 1(1):67–82. <https://doi.org/10.1109/4235.585893>
- Yang FJ (2019) An extended idea about decision trees. In: 2019 International Conference on Computational Science and Computational Intelligence (CSCI), pp 349–354, <https://doi.org/10.1109/CSCI49370.2019.00068>
- Yanyang Z, Zhang D, Gong Y (2022) Eliminating the high false-positive rate in defect prediction through bayesnet with adjustable weight. *Expert Systems* 39. <https://doi.org/10.1111/exsy.12977>
- Zhang X, Ni X, Deng Y, et al (2021) Chinese license plate recognition using machine and deep learning models. In: 2021 IEEE 2nd International Conference on Pattern Recognition and Machine Learning (PRML), pp 342–346, <https://doi.org/10.1109/PRML52754.2021.9520386>
- Zimmermann T, Premraj R, Zeller A (2007) Predicting defects for eclipse. In: Third International Workshop on Predictor Models in Software Engineering (PROMISE’07: ICSE Workshops 2007), pp 9–9, <https://doi.org/10.1109/PROMISE.2007.10>